Bitcoin Script

Dr. Sourav Kanti Addya

Cloud and Smart System Services Lab
Department of Computer Science and Engineering
National Institute Of Technology Karnataka, Surathkal

Cloud @ Department of CSE

- Department offers Elective courses for UG and PG
  - Cloud Computing
  - Cloud Security
  - Next Generation Cloud Architecture
- Cloud and Smart System Services Lab
  - **Working on different verticals:** Cloud System, IaaS, Fog, Edge, Vehicular-Cloud
  - **Dedicated High End server:** Scholars working with **Virtual machine** and **containers**.
  - Physical Edge-Fog Setup
  - **Visit:** https://cs3-lab.github.io/

CS 3
CLOUD AND SMART SYSTEM SERVICES LAB

Outline

- Motivation: Why Bitcoin Script Exists
- What is Bitcoin Script?
- How Script Works
- High-Level Transaction Structure
- The Stack Model (Bitcoin Script Execution)
- Standard Scripts
- p2pk: Pay-to-pubkey
- p2pkh: Pay-to-pubkey-hash
- p2sh: Pay-to-script-hash

Introduction to Bitcoin Script

Motivation: Why Bitcoin Script Exists

- When you send Bitcoin, the network *must verify*:
  - Are you really the owner?
  - Are you allowed to spend these coins?

- Bitcoin needs a way to define:
  "*Under what conditions can this money be spent?*"

- For example:
  - Provide a valid digital signature
  - Get approval from multiple people
  - Wait until a specific time

- Instead of hard-coding one rule, Bitcoin uses:
  - ➜ Bitcoin Script — a small language to define spending conditions

CLOUD AND SMART SYSTEM SERVICES LAB

What is Bitcoin Script?

- In Bitcoin, Script is the smart contract language, or the programming language used to express *the conditions* under which bitcoins are spendable.
- Bitcoin has the digital equivalent of a contract in Script.
- Script is a *stack-based language* similar to Forth. It's intentionally limited in the sense that it avoids certain features.
- Specifically, Script avoids any mechanism for loops and is therefore *not Turing complete*.
- Transactions assign bitcoins to a locking script. You can think of this as a lockbox where some money is deposited that only a particular key can open. The money inside, of course, can only be accessed by the owner who has the key. this proves ownership of the locked box,
  which authorizes spending of the funds.

Why Bitcoin Isn't Turing Complete

- Anyone can create a Script program that every full node on the network executes. If Script were Turing complete, it would be possible for the loop to go on executing forever. This would cause validating nodes to enter and never leave that loop.
- This would be an easy way to attack the network through what would be called a *denial-of-service (DoS) attack*. A single Script program with an infinite loop could take down Bitcoin! This would be a large systematic vulnerability, and protecting against this vulnerability is one of the major reasons why Turing completeness is avoided.

Why Bitcoin Isn't Turing Complete

- Anyone can create a Script program that every full node on the network executes. If Script were Turing complete, it would be possible for the loop to go on executing forever. This would cause validating nodes to enter and never leave that loop.
- This would be an easy way to attack the network through what would be called a *denial-of-service (DoS) attack*. A single Script program with an infinite loop could take down Bitcoin! This would be a large systematic vulnerability, and protecting against this vulnerability is one of the major reasons why Turing completeness is avoided.

How Script Works

- Script is a programming language, and like most programming languages, it processes one command at a time. The commands operate on a stack of elements.
- There are two possible types of commands: elements and operations.
- Elements are data. Technically, processing an element pushes that element onto the stack. Elements are byte strings of length 1 to 520.
- A typical element might be a DER signature or a SEC pubkey

How Script Works

Operations do something to the data. They consume zero or more elements from the processing stack and push zero or more elements back to the stack.

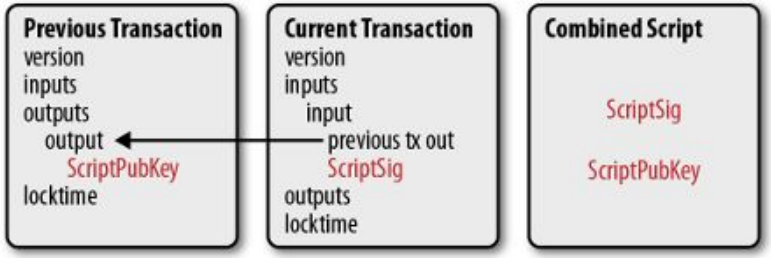
After all the commands are evaluated, the top element of the stack must be nonzero for the script to resolve as valid. Having no elements in the stack or the top element being 0 would resolve as invalid. Resolving as invalid means that the transaction that includes the unlocking script is not accepted on the network

High-Level Transaction Structure

- The Script object represents the command set that requires evaluation. To evaluate a script, we need to combine the ScriptPubKey and ScriptSig fields.
- The *lockbox (ScriptPubKey)* and the *unlocking mechanism (ScriptSig)* are in different transactions.
- Specifically, the lockbox is where the bitcoins are received, and the unlocking script is where the bitcoins are spent. The input in the spending transaction points to the receiving transaction

High-Level Transaction Structure

- Since the ScriptSig unlocks a ScriptPubKey, we need a mechanism by which the two scripts combine. To evaluate the two together, we take the commands from the ScriptSig and ScriptPubKey and combine them as in Figure.



- The commands from the ScriptSig *go on top of all the commands from the ScriptPubKey*. Instructions are processed one at a time until no commands are left to be processed (or the script fails)

The Stack Model (Bitcoin Script Execution)

- Bitcoin Script uses a *stack-based execution model*
- A stack works on *Last-In, First-Out (LIFO)* principle
    - The last value added is the first one removed
- How it works:
    - Data values are pushed onto the stack
    - Operations (opcodes) use values from the stack
    - Results are placed back onto the stack

The Stack Model (Bitcoin Script Execution)

- A script can be any arbitrary program. Script is a smart contract language and can lock bitcoins in many different ways. Figure 1 is an example ScriptPubKey.

```
55935987

55 - OP_5
93 - OP_ADD
59 - OP_9
87 - OP_EQUAL
```

Example ScriptPubKey

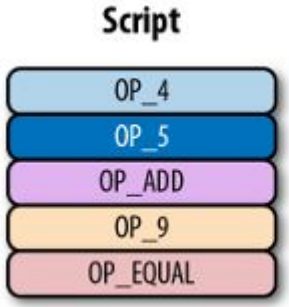
```
54

54 - OP_4
```

Example ScriptSig

Figure 2 is a ScriptSig that will unlock the the ScriptPubKey

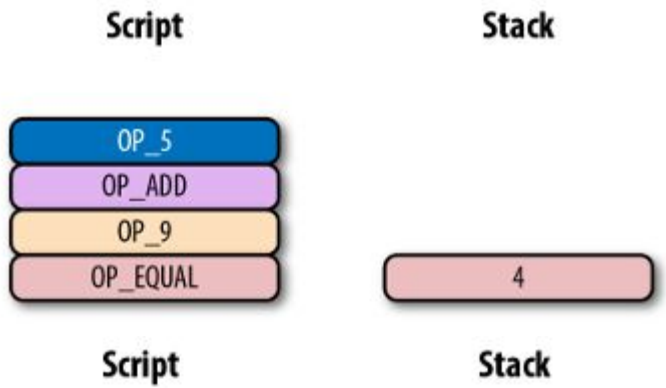
The Stack Model (Bitcoin Script Execution)

Script Combined



Script Evaluation using Stack

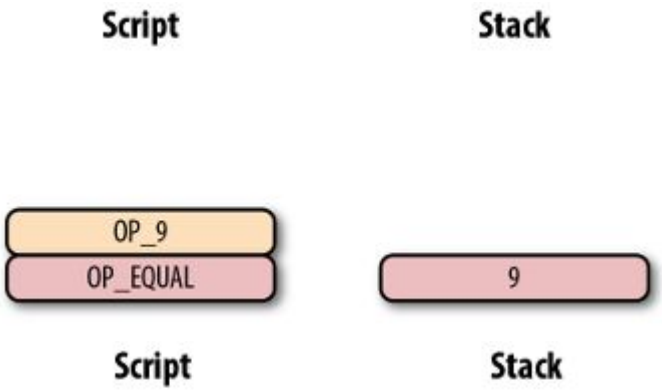
The Stack Model (Bitcoin Script Execution)



OP_4 will push a 4 to the stack

OP_5 will likewise push a 5 to the stack

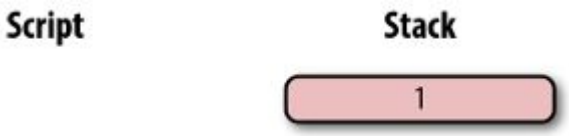
The Stack Model (Bitcoin Script Execution)



OP_ADD will consume the top two elements of the stack, add them together, and push the sum to the stack

OP_9 will push a 9 to the stack

The Stack Model (Bitcoin Script Execution)



OP_EQUAL will consume two elements and push a 1 if they're equal and a 0 if no

Note that the ScriptSig here isn't particularly hard to figure out and contains no signature. As a result, the ScriptPubKey is vulnerable to being taken by anyone who can solve it. Think of this ScriptPubKey as a lockbox with a very flimsy lock that anyone can break into. It is for this reason that most transactions have a signature requirement in the ScriptSig.

Short Exercise

Create a ScriptSig that can unlock this ScriptPubKey:

767695935687

Note that OP_MUL multiplies the top two elements of the stack.

- 56 = OP_6
- 76 = OP_DUP
- 87 = OP_EQUAL
- 93 = OP_ADD
- 95 = OP_MUL

Standard Scripts

There are many types of standard scripts in Bitcoin, including the following:

- p2pk: Pay-to-pubkey
- p2pkh: Pay-to-pubkey-hash
- p2sh: Pay-to-script-hash

Addresses are known script templates like these. Wallets know how to interpret various address types (p2pkh, p2sh, p2wpkh) and create the appropriate ScriptPubKeys. All of these scripts here have a particular type of address format (Base58, Bech32) so wallets can pay to them.

P2PK (Pay-to-Public-Key)

- One of the original script formats used in Bitcoin is P2PK; many early coins, including those believed to belong to Satoshi Nakamoto, were stored using P2PK outputs.
- **Direct Payment to a Public Key:**
  Bitcoins are sent directly to a recipient's public key rather than to an address derived from it.
- **Locked by ScriptPubKey**:
  The transaction output contains a ScriptPubKey that specifies the required public key, placing the funds under the control of the corresponding private key holder.
- **Spending via ECDSA Signature:**
  To unlock and spend the bitcoins, the owner must provide a valid ECDSA signature consisting of values (r, s), along with the message hash (z) and the public key (P) for verification.

P2PK (Pay-to-Public-Key)

P2PK Script Structure

- **ScriptPubKey (Locking Script)**: Defines where the bitcoins go and acts as a “lockbox.”
  Format:

  ```
  <Public Key> OP_CHECKSIG
  ```

  The presence of OP_CHECKSIG is critical—it verifies the signature against the specified public key.

- **Public Key Format:**
  The public key embedded in the ScriptPubKey can be compressed or uncompressed.
  In early Bitcoin usage, the uncompressed format was primarily used.
- **ScriptSig (Unlocking Script):** Used to spend the locked bitcoins.
  Format:

  ```
  <Signature>
  ```

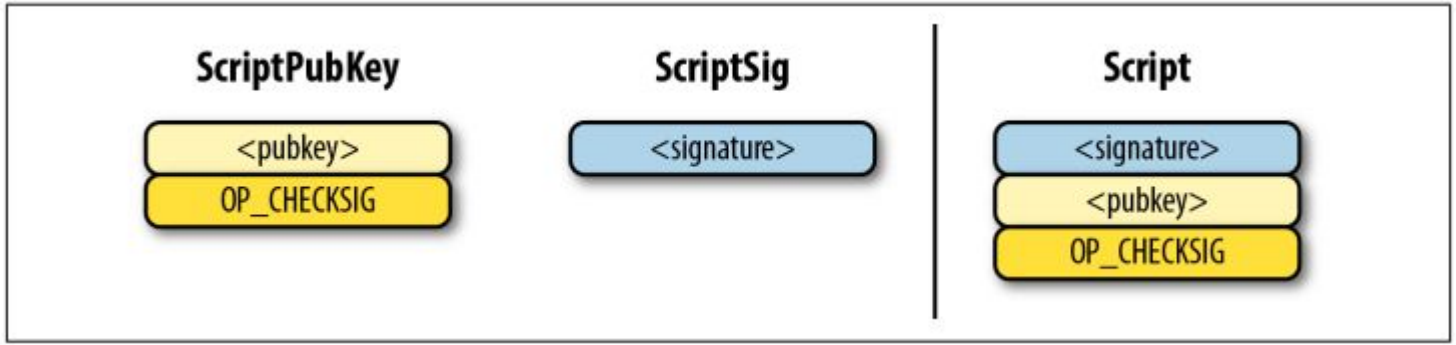
  The signature proves ownership of the corresponding private key.

CLOUD AND SMART SYSTEM SERVICES LAB

P2PK (Pay-to-Public-Key)

P2PK Script Structure-Execution Flow:

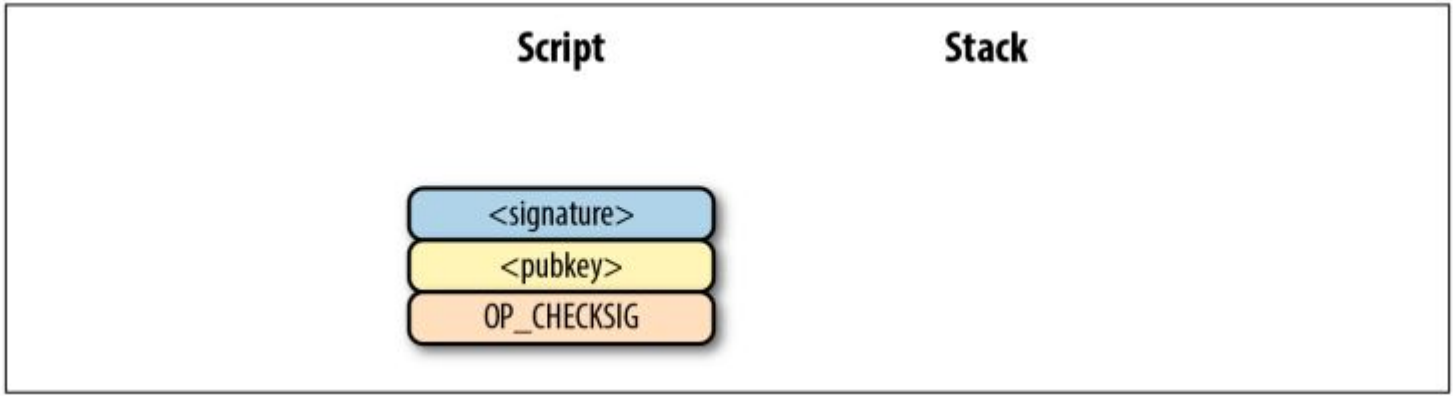
When ScriptSig and ScriptPubKey are executed together, the signature is checked by OP_CHECKSIG. If the signature is valid for the provided public key and transaction data, the funds can be spent.



P2PK (Pay-to-Public-Key)

P2PK Script Structure-Execution Flow:

Stack-Based Evaluation Model: Script commands (left column) are processed one at a time, affecting the stack (right column). For the script to be valid, the top stack element must be nonzero at the end of execution.

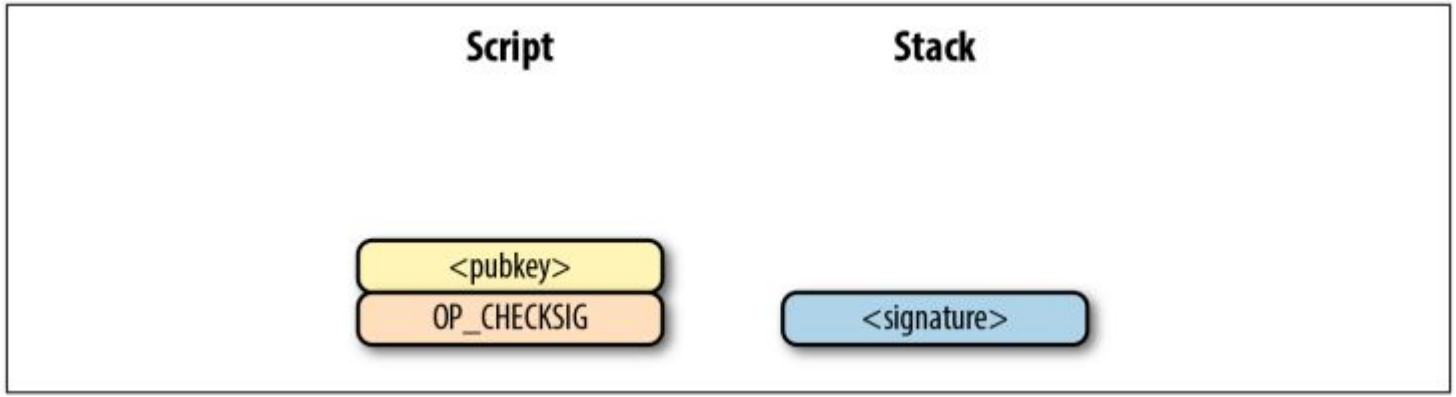


P2PK (Pay-to-Public-Key)

Step 1 – Push Signature:

The first command (from ScriptSig) is the signature + sighash byte.

This data element is pushed onto the stack.

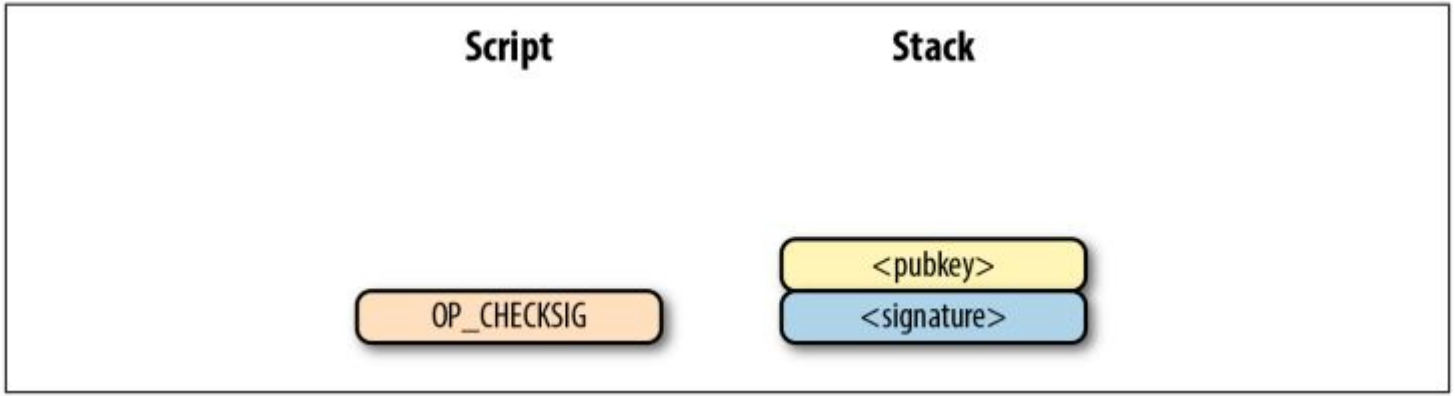


P2PK (Pay-to-Public-Key)

Step 2 – Push Public Key:

The next command (from ScriptPubKey) is the public key.

This is also pushed onto the stack.

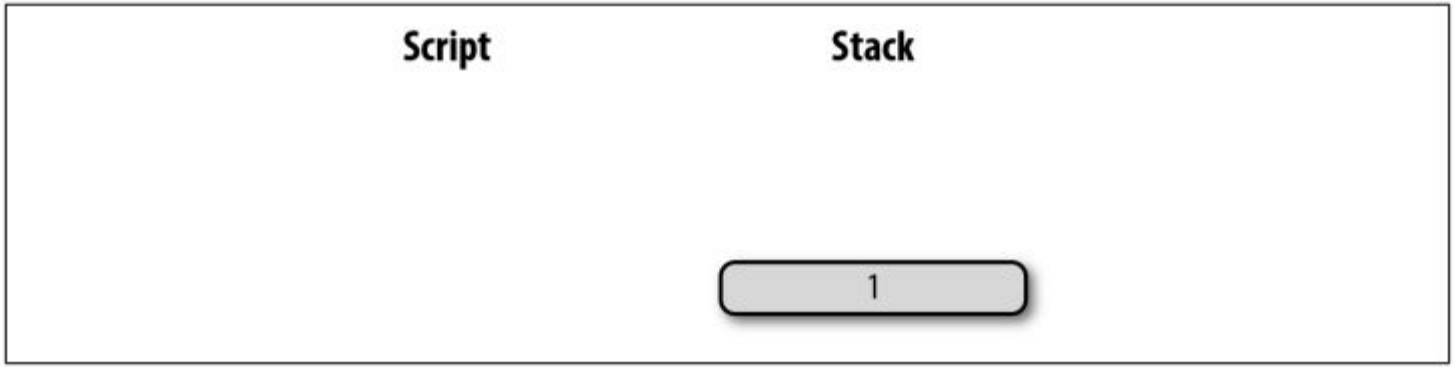


P2PK (Pay-to-Public-Key)

Step 3 – OP_CHECKSIG Execution:

OP_CHECKSIG pops the public key and signature, verifies them against the transaction, and:

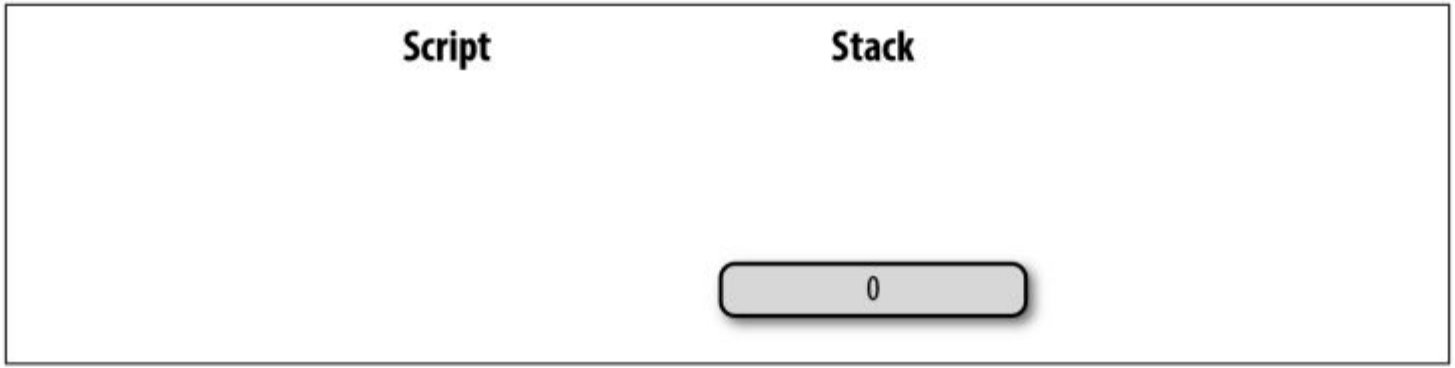
- Pushes 1 if the signature is valid
- Pushes 0 if invalid



P2PK (Pay-to-Public-Key)

Final Validation Rule:

If the top stack element is 1 (nonzero) → Script is valid → Transaction input is accepted.
If the top stack element is 0 → Script fails → Transaction is invalid.
Only someone with the correct private key can produce a valid signature that passes OP_CHECKSIG.



P2PKH (Pay-to-Public-Key-Hash)

- **Improved Alternative to P2PK:**
  P2PKH became the standard Bitcoin transaction type after 2010 due to better efficiency and added security, while older P2PK outputs remain supported.
- **Two Key Advantages:**
  - Shorter Addresses
  - Additional Hash Protection (SHA-256 followed by RIPEMD-160)
- **Hash160 Mechanism:**
  The public key is hashed using SHA-256, then RIPEMD-160.
  This combined process is called hash160, producing a 160-bit (20-byte) value.
- **Address Encoding:**
  The 20-byte hash160 result is encoded into a Base58Check Bitcoin address

P2PKH (Pay-to-Public-Key-Hash)

P2PKH Script Structure

- **ScriptPubKey (Locking Script):**

  Instead of storing the public key, it stores its 20-byte hash (hash160).

  ```
  OP_DUP OP_HASH160 <hash> OP_EQUALVERIFY OP_CHECKSIG
  ```

- **Key Opcodes:**
  - OP_HASH160 → hashes the provided public key
  - OP_EQUALVERIFY → checks the hash matches the expected value
  - OP_CHECKSIG → verifies the signature
- **ScriptSig (Unlocking Script):**

  Contains both the DER signature and the public key.

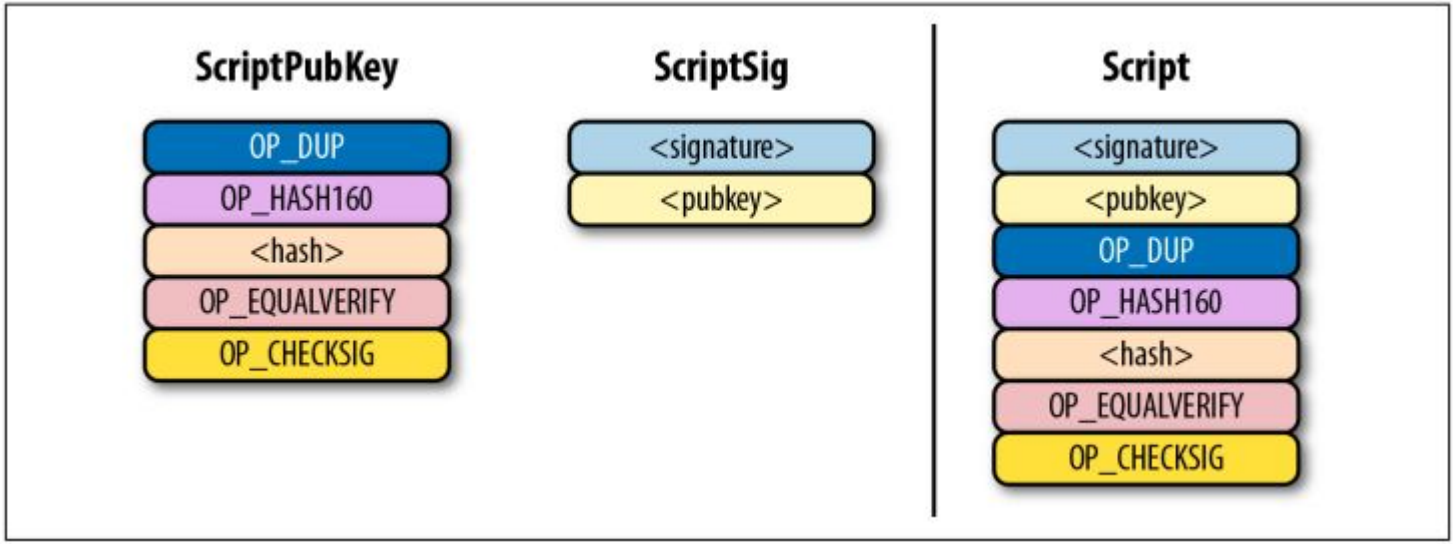
  ```
  <Signature> <Public Key>
  ```

CLOUD AND SMART SYSTEM SERVICES LAB

P2PKH (Pay-to-Public-Key-Hash)

P2PKH Script Structure-Execution Flow:

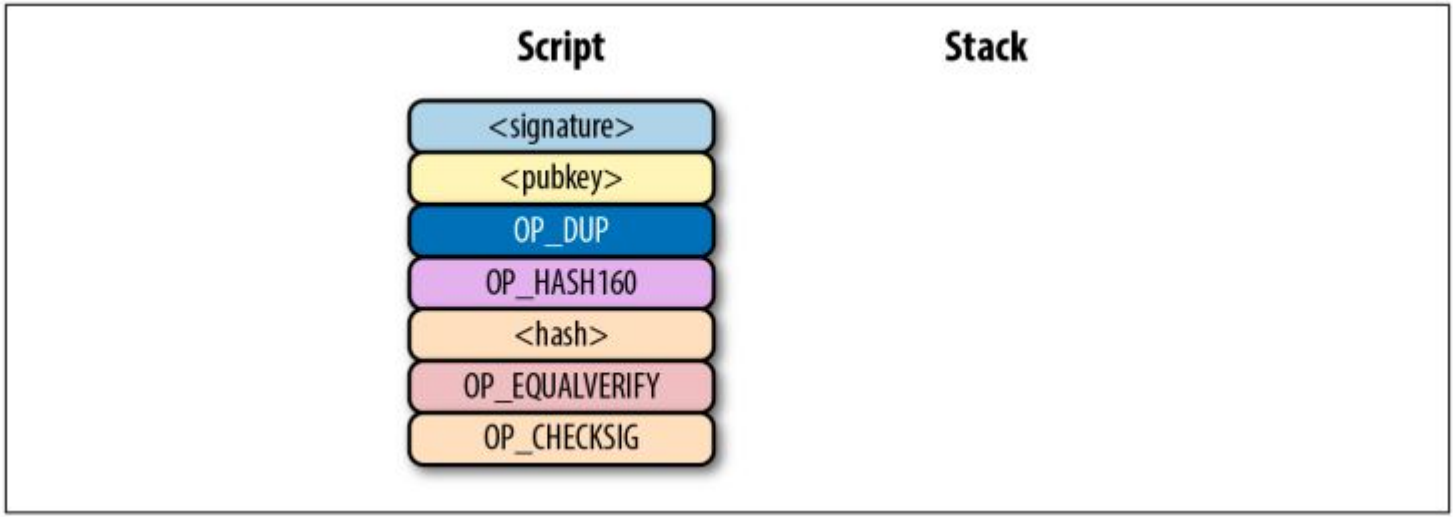
The ScriptPubKey and ScriptSig combine to form a list of commands that look as below. At this point, the script is processed one command at a time.



P2PKH (Pay-to-Public-Key-Hash)

P2PKH Script Structure-Execution Flow:

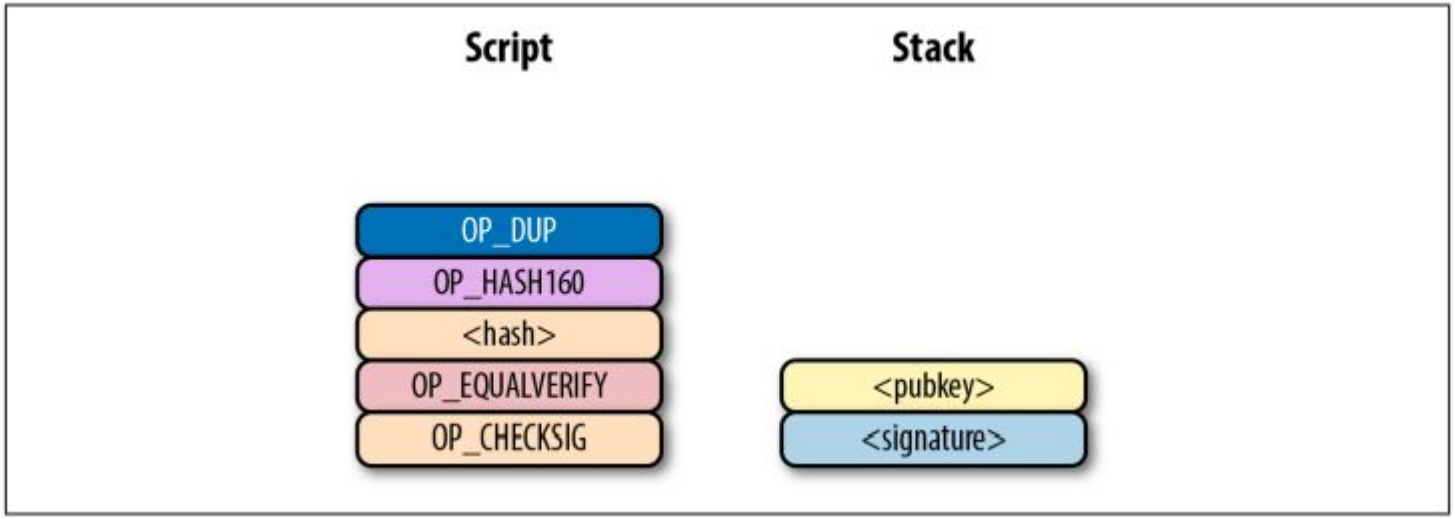
We start with the commands as combined in Figure below.



P2PKH (Pay-to-Public-Key-Hash)

P2PKH Script Structure-Execution Flow:

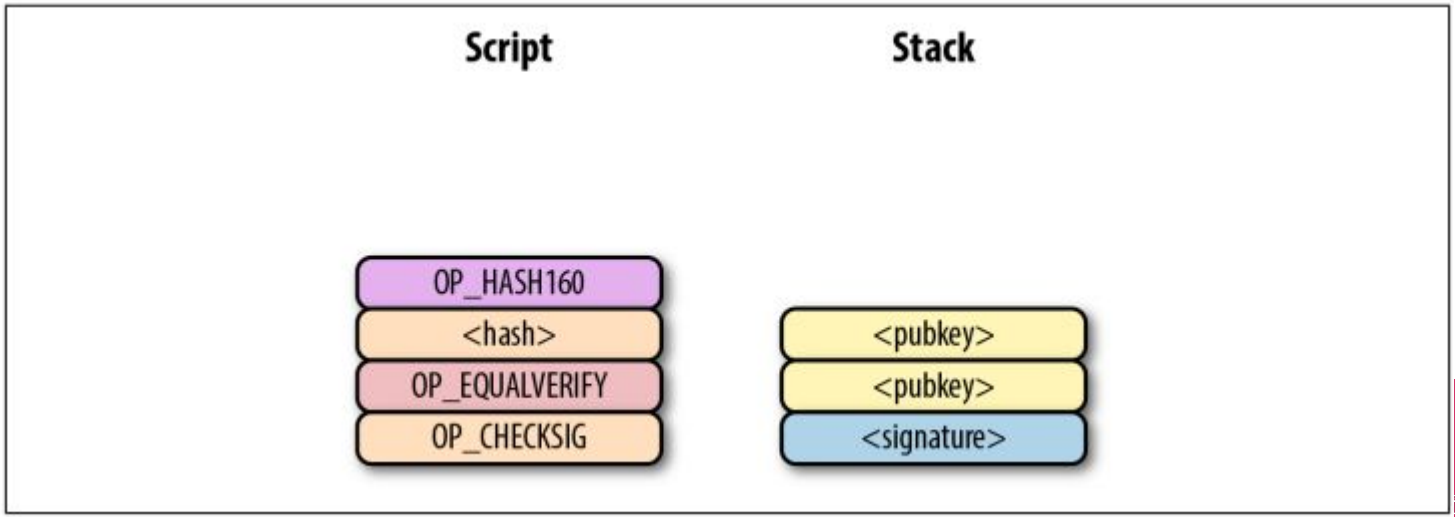
The first two commands are elements, so they are pushed to the stack



P2PKH (Pay-to-Public-Key-Hash)

P2PKH Script Structure-Execution Flow:

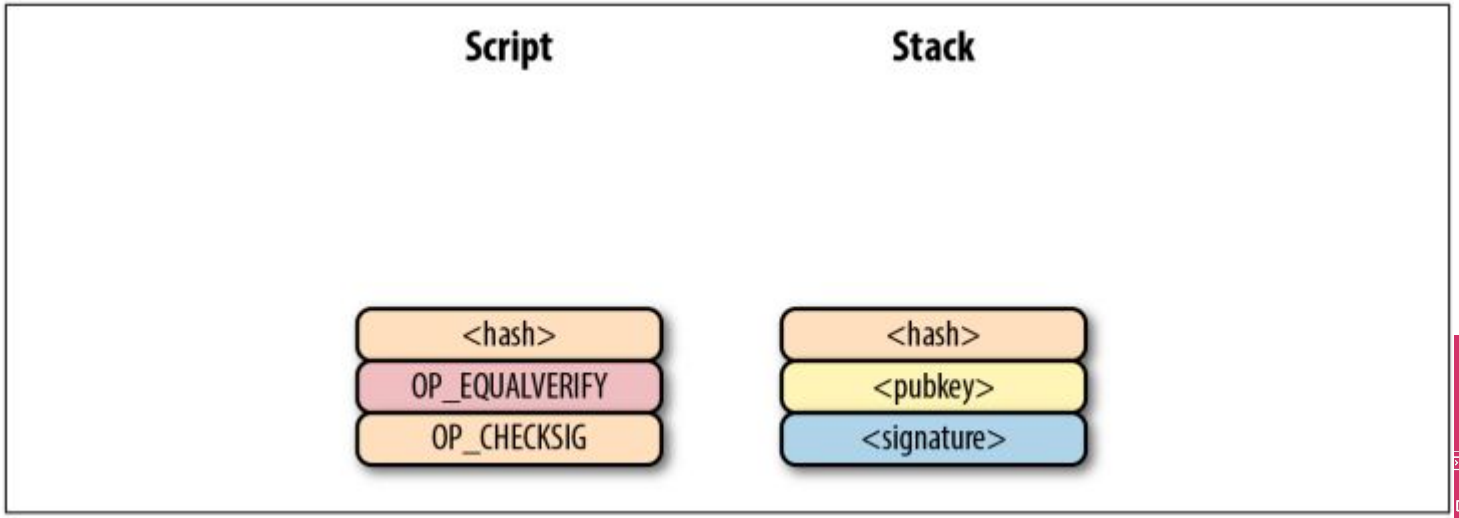
OP_DUP duplicates the top element, so the pubkey gets duplicated.



P2PKH (Pay-to-Public-Key-Hash)

P2PKH Script Structure-Execution Flow:

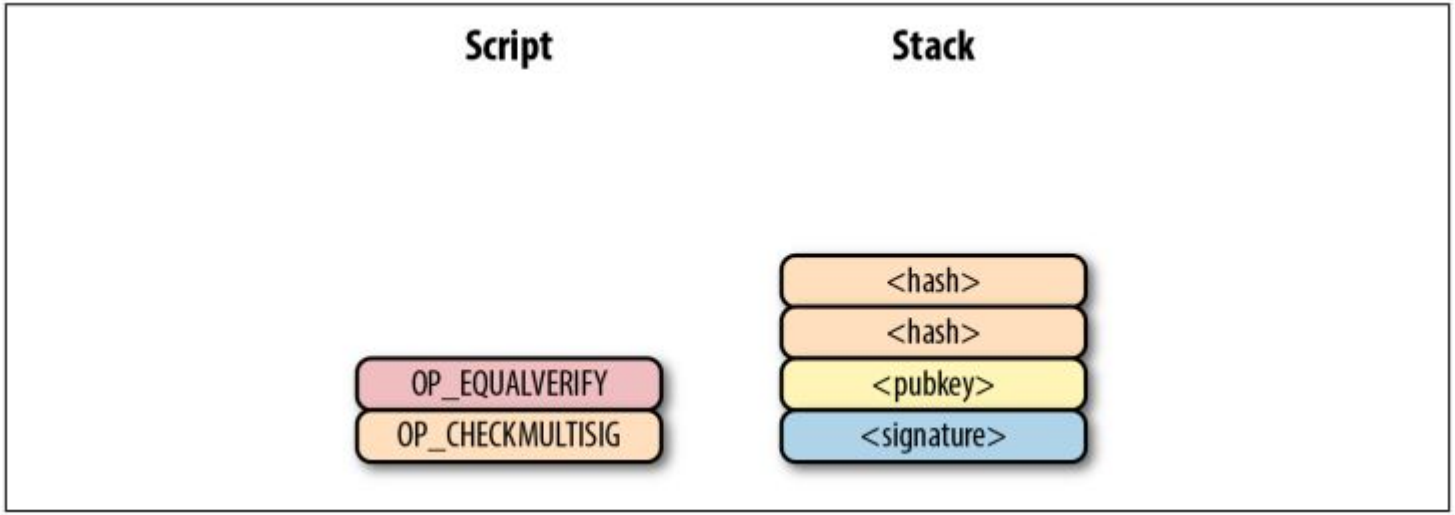
OP_HASH160 takes the top element and performs the hash160 operation on it (sha256 followed by ripemd160), creating a 20-byte hash



P2PKH (Pay-to-Public-Key-Hash)

P2PKH Script Structure-Execution Flow:

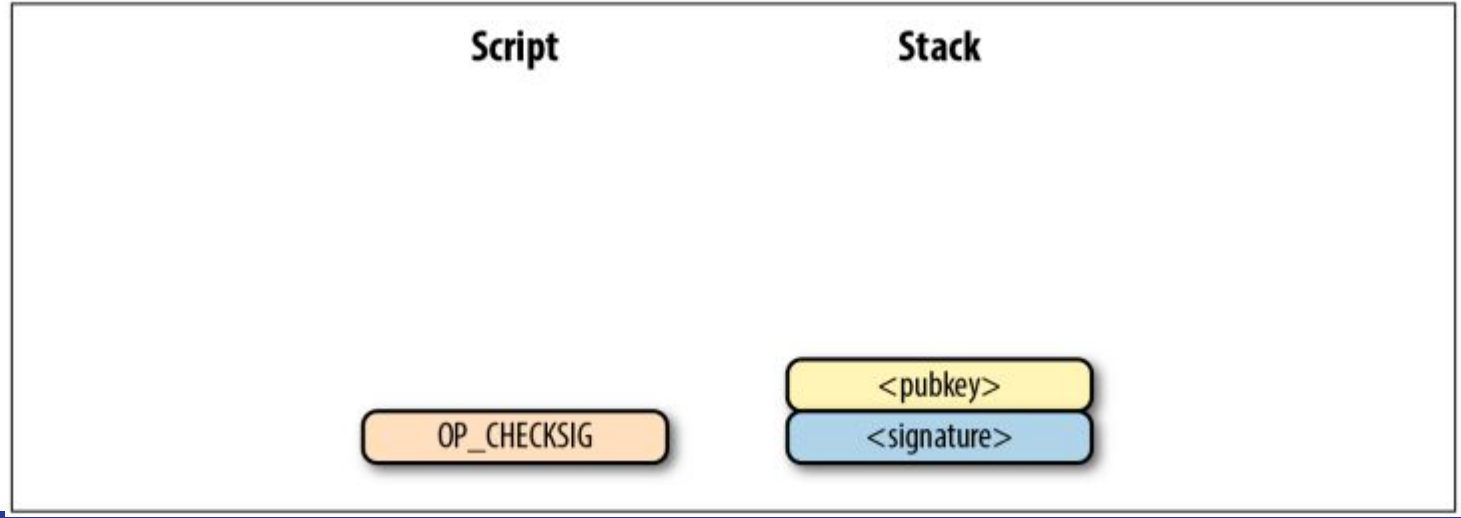
The 20-byte hash is an element and is pushed to the stack



P2PKH (Pay-to-Public-Key-Hash)

P2PKH Script Structure-Execution Flow:

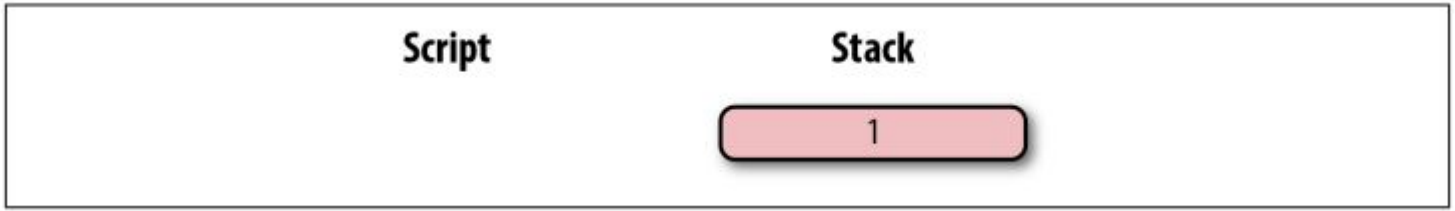
We are now at OP_EQUALVERIFY. This opcode consumes the top two elements and checks if they're equal. If they are equal, the script continues execution. If they are not equal, the script stops immediately and fails. We assume here that they're equal



P2PKH (Pay-to-Public-Key-Hash)

P2PKH Script Structure-Execution Flow:

We are now exactly where we were during the OP_CHECKSIG part of processing p2pk. Once again, we assume that the signature is valid



Two Failure Conditions:

- Public key does not hash to the expected 20-byte value.
- Signature is invalid even if the hash matches.

P2SH (Pay-to-Script-Hash)

What is Multisig?

Multisig (multiple signatures) requires more than one private key to authorize spending.This adds security by preventing a single key from controlling funds, enabling shared control and flexible spending rules.

What is P2SH (Pay-to-Script-Hash)?

- Complex scripts like multisig create very long ScriptPubKeys, which are inefficient and bulky.
- P2SH solves this by storing only the hash of the complex script in the ScriptPubKey.
- The full script (redeem script) is only revealed and validated when spending the funds.
- Benefits of P2SH:
  - Keeps locking scripts short and efficient.
  - Improves privacy by hiding script details until necessary.
  - Makes using complex spending conditions easier and more practical.

CLOUD AND SMART SYSTEM SERVICES LAB

P2SH (Pay-to-Script-Hash)

P2SH Script Structure

- **ScriptPubKey (Locking Script):**
  Contains the hash160 of the RedeemScript instead of the full script:

  ```
  OP_HASH160 <hash160(RedeemScript)> OP_EQUAL
  ```

  Locks the funds to the hash of a script, keeping the locking script short.
  The full RedeemScript is not revealed until spending.

- **ScriptSig (Unlocking Script):**

  Provides the following during redemption:
  - Signatures required by the RedeemScript (e.g., for multisig).
  - The RedeemScript itself (revealed only at spending).

  Execution verifies that the hash of the provided RedeemScript matches the hash in the ScriptPubKey. Only then are the commands in the RedeemScript executed to validate the signatures.

P2SH (Pay-to-Script-Hash)

P2SH Script Structure

- **RedeemScript**
  The actual script that defines the spending rules

```
OP_m <pubkey1> ... <pubkeyn> OP_n OP_CHECKMULTISIG
```

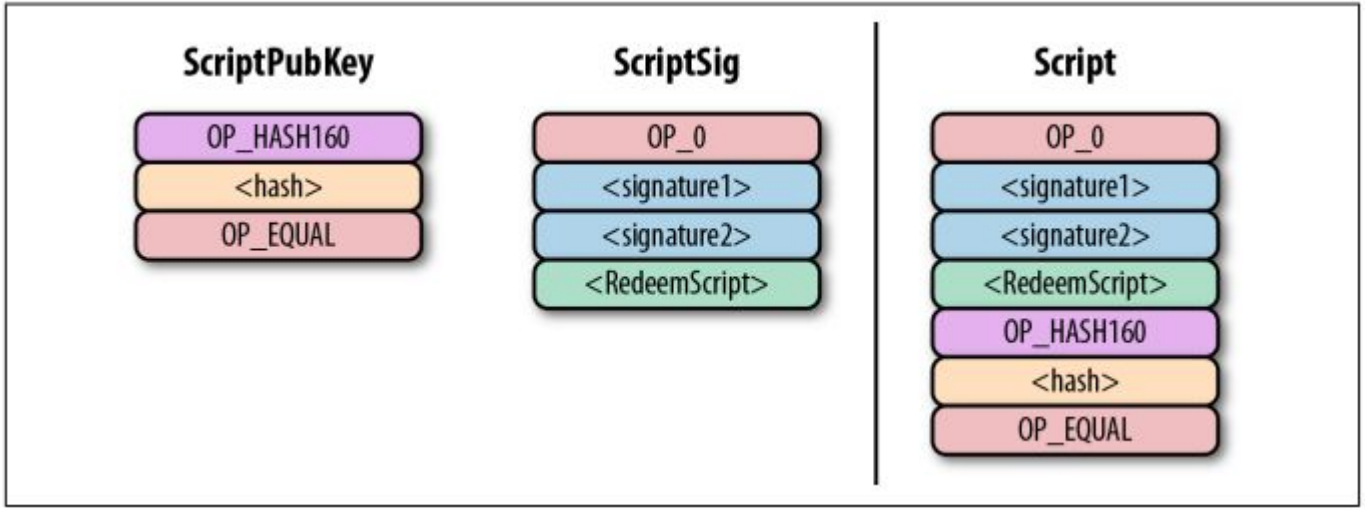
Must be securely stored by the creator of the P2SH address.

Lost RedeemScripts = lost funds, since the blockchain only stores the hash.

P2SH (Pay-to-Script-Hash)

P2SH Script Structure-Execution Flow:

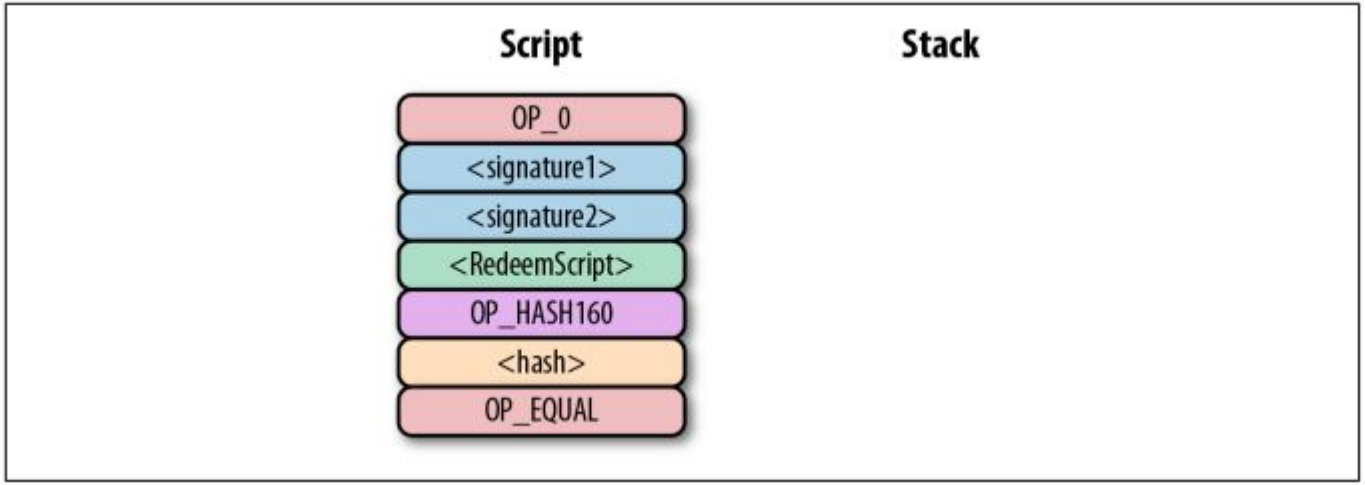
Figure below shows how the ScriptSig and ScriptPubKey combine.



P2SH (Pay-to-Script-Hash)

P2SH Script Structure-Execution Flow:

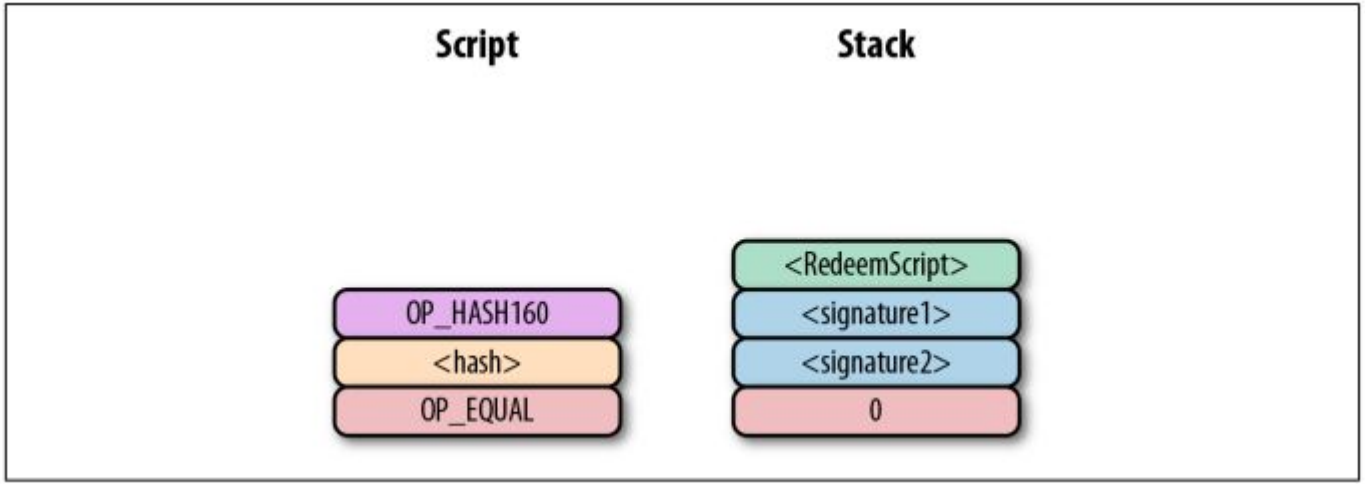
We start with the Script commands



P2SH (Pay-to-Script-Hash)

P2SH Script Structure-Execution Flow:

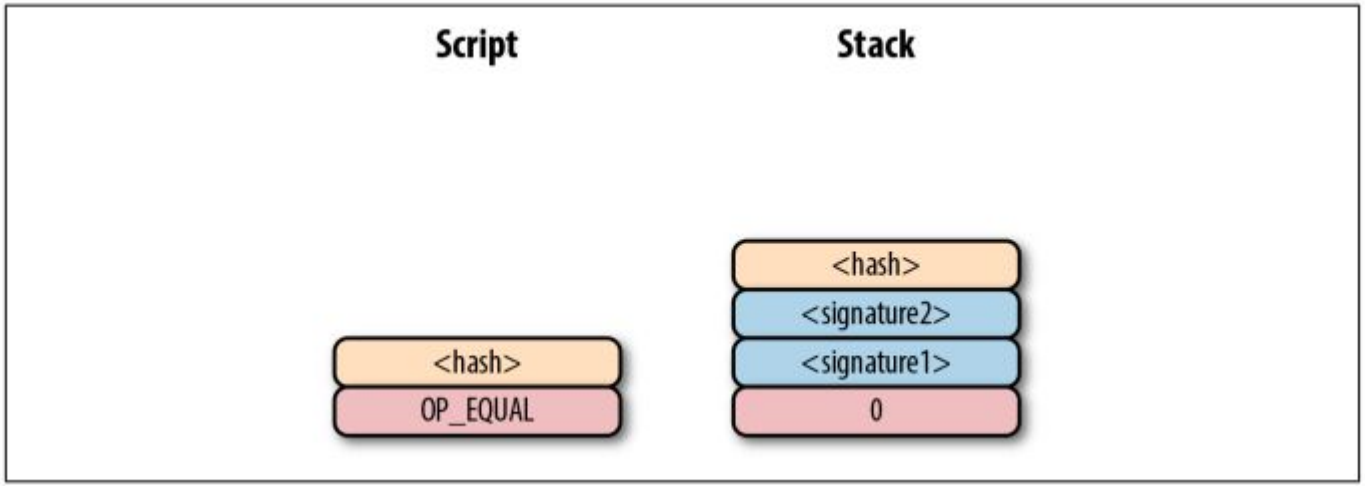
OP_0 will push a 0 to the stack, and the two signatures and the RedeemScript will be pushed to the stack directly



P2SH (Pay-to-Script-Hash)

P2SH Script Structure-Execution Flow:

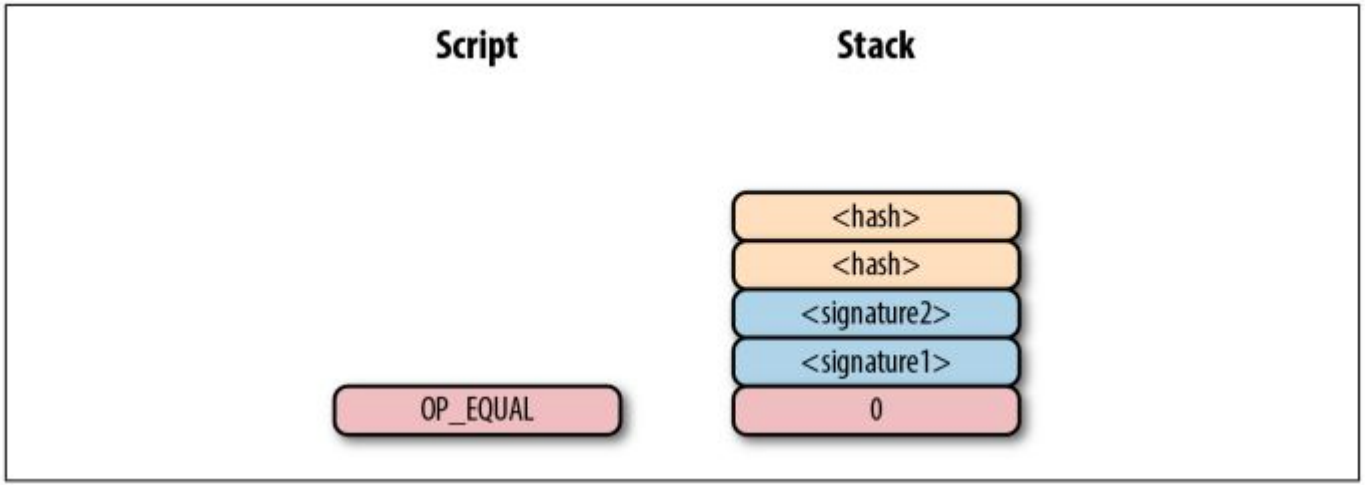
OP_HASH160 will hash the RedeemScript, which will make the stack look like



P2SH (Pay-to-Script-Hash)

P2SH Script Structure-Execution Flow:

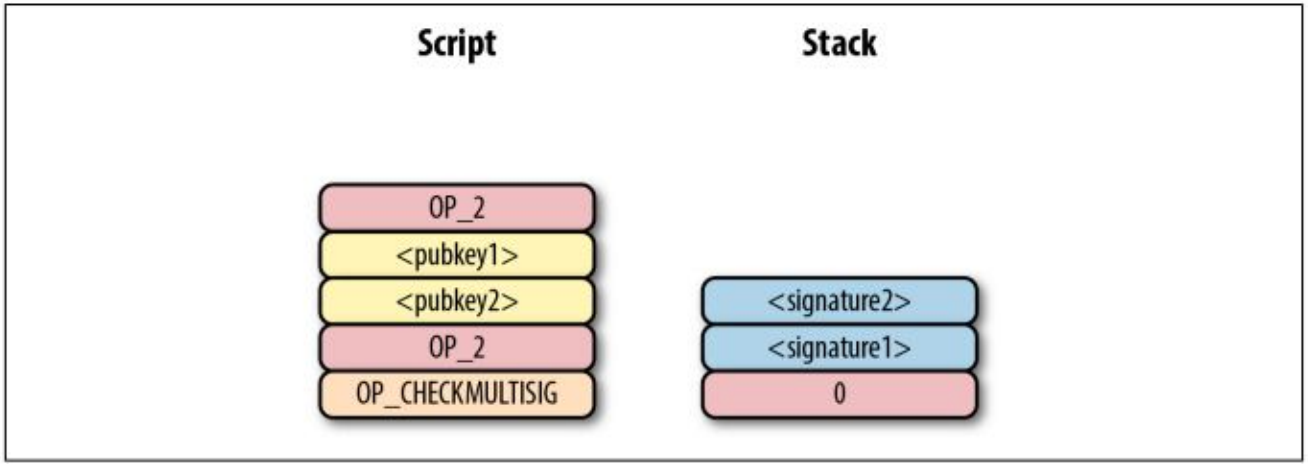
The 20-byte hash will be pushed to the stack



P2SH (Pay-to-Script-Hash)

P2SH Script Structure-Execution Flow:

And finally, OP_EQUAL will compare the top two elements. If the hashes match, the RedeemScript is parsed and executed as new Script commands.

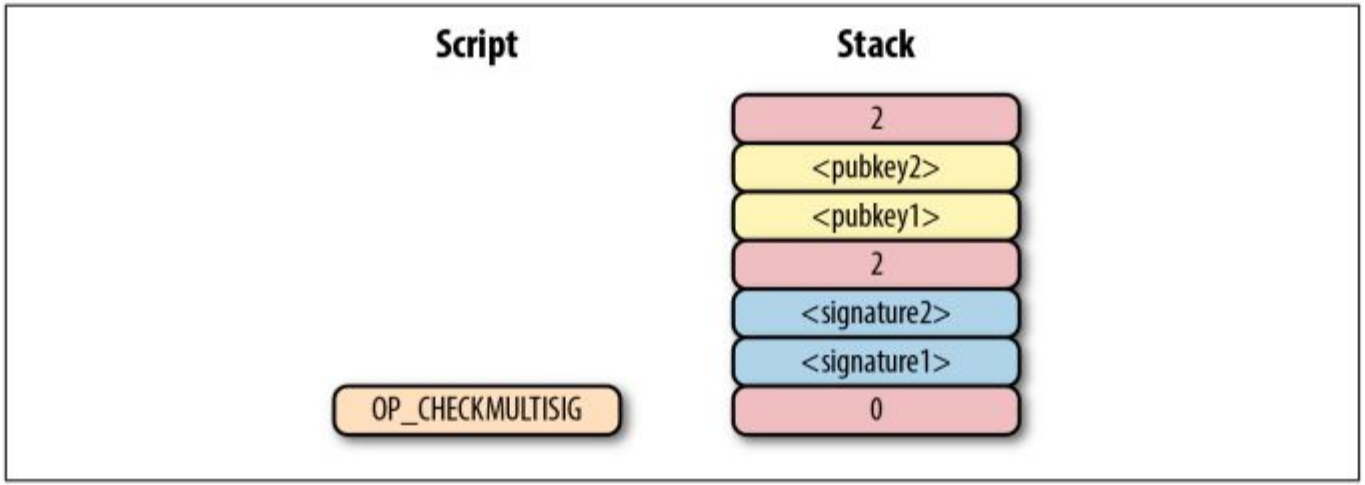


P2SH first verifies the hash of the RedeemScript, then executes the actual spending conditions contained inside it.

P2SH (Pay-to-Script-Hash)

P2SH Script Structure-Execution Flow:

OP_2 pushes a 2 to the stack, the pubkeys are also pushed, and a final OP_2 pushes another 2 to the stack



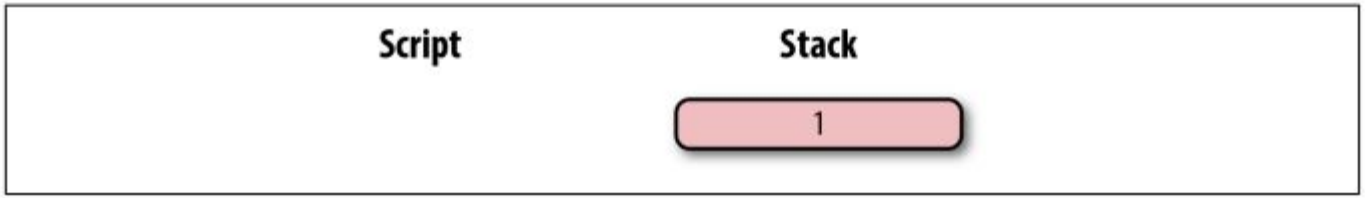
P2SH (Pay-to-Script-Hash)

P2SH Script Structure-Execution Flow:

OP_CHECKMULTISIG verifies that the required signatures are valid.

Final Result:

- If enough valid signatures are provided → stack ends with 1 → transaction is valid.
- If verification fails → stack ends with 0 →transaction is invalid.



References

- https://en.bitcoin.it/wiki/Script
- Programming Bitcoin:Learn How to Program Bitcoin from Scratch - Book by Jimmy Song

  https://digi-lib.stekom.ac.id/assets/dokumen/ebook/feb_d82be9cf1cb52e2b294a82275318a5c8235444eb_1654093256.pdf

Thank You

Cloud and Smart System Services Lab (CS^3)

https://souravkaddya.in/

CLOUD AND SMART SYSTEM SERVICES LAB

CLOUD AND SMART SYSTEM SERVICES LAB

Sourav Kanti Addya, Ph.D.

Assistant Professor(Grade-I), Dept of CSE

National Institute of Technology Karnataka, Surathkal, India

Email: souravkaddya@nitk.edu.in

Visit: https://souravkaddya.in/
https://cs3-lab.github.io/